



Agent Atlas

Agent 技术完整知识树：从哪里来，当下在哪里，未来去哪里

从哪里来

当下在哪里

未来去哪里

写在前面

Agent 已从“让模型调用一个工具”的演示，进入系统工程阶段。本书不把 Agent 等同于聊天模型，也不以框架热度组织内容。它沿历史、机制、架构、生命周期与未来五个层次，回答技术从哪里来、当下真正成熟到什么程度、下一阶段要解决什么。

阅读时请持续追问四件事：目标是否可验证，行动是否受约束，轨迹是否可观察，失败是否可恢复。缺少其中任意一项，能力越强，系统风险可能越大。

目录 1/4

第一章 什么是 Agent	7
定义与边界	8
核心术语表	9
成熟度模型	10
阅读路径	11
Agent 分类法	12
第二章 Agent 从哪里来	13
Foundation Models 转折	14
历史时间线	15
LLM Agent 里程碑	16
规划、搜索与控制	17
强化学习遗产	18
符号智能体与 BDI	19
第三章 Agent 的解剖	20
Agent Loop 与状态	21
上下文工程	22
目标、任务与环境	23
人机协作	24
学习与适应	25
记忆	26
多模态与 Computer Use	27
推理、规划与反思	28
检索与 Grounding	29
工具与 Function Calling	30
第四章 Agent 架构	31
反模式	32
图与事件驱动 Agent	33
多 Agent 模式	34

目录 2/4

生产参考架构	35
路由、Handoff 与编排	36
Sandbox 与运行时	37
单 Agent 模式	38
状态与持久执行	39
Workflow 与 Agent	40
Agent-to-Agent 协议	41
Agent-UI 协议	42
发现、身份与授权	43
Model Context Protocol	44
可移植性与兼容性	45
第五章 从原型到生产	46
01 发现与定界	47
02 设计与威胁建模	48
03 构建与集成	49
04 测试、评测与 Red Team	50
05 部署与发布	51
06 观测与运营	52
07 改进与优化	53
08 退役与归档	54
第六章 评测与可观测	55
Benchmark 全景	56
能力与可靠性	57
成本、延迟与质量	58
数据集与测试用例	59
评测策略	60
人工评测	61
安全与对抗评测	62

目录 3/4

轨迹与工具使用评测	63
第七章 安全、安全性与治理	64
Agent 威胁模型	65
Guardrails 与人类控制	66
身份、委托与供应链	67
Agent 事件响应	68
记忆与数据安全	69
权限与最小特权	70
隐私、审计与合规	71
Prompt Injection	72
沙箱与隔离	73
第八章 生态与应用	74
部署基础设施	75
框架与 SDK	76
记忆与检索系统	77
模型与提供商	78
可观测与评测工具	79
运行时与 Agent 平台	80
技术选型指南	81
技术全景	82
工具、连接器与 MCP	83
浏览器与 Computer Use Agents	84
Coding Agents	85
客户运营 Agents	86
企业工作流 Agents	87
研究、数据与科学 Agents	88
机器人与 Embodied Agents	89
语音与多模态 Agents	90

目录 4/4

第九章 2026：我们在哪里

2026-09-30 快照

能力前沿

经济性与运营模型

局限与瓶颈

生产成熟度

第十章 Agent 将去向何处

自适应与自我改进 Agent

自主性与人类控制

未来 3-5 年

多 Agent 生态

未来 12-24 个月

开放研究问题

情景与路标

标准与基础设施

第十一章 工程行动手册

Build or Buy 决策指南

案例分析模板

失败图谱

学习路径

生产就绪检查表

参考蓝图

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

CHAPTER 01

第一章 什么是 Agent

Agent 是一个受约束的目标驱动闭环，而不是某种特定模型或 SDK。它读取状态、选择行动、使用工具改变环境、观察结果，并在成功、失败、求助或预算耗尽时停止。Workflow 与 Agent 的差别在控制路径：前者预定义，后者允许模型动态决定。

工程上的关键选择不是“要不要更自主”，而是哪些判断值得动态化、哪些边界必须确定性执行。

stable

定义与边界

用自主决策、环境行动和闭环反馈区分 Agent、Workflow 与 Chatbot。

Agent 是在明确约束内，为完成目标而自主选择下一步、调用工具改变环境、观察结果并调整行动的系统。核心不是“用了大模型”，而是存在 目标 - 决策 - 行动 - 观察 - 终止 闭环。

Chatbot 主要生成一次响应；Workflow 的控制路径由代码或图预先规定；Agent 允许模型在运行时决定路径。真实产品常是混合体：可预测部分使用 workflow，信息不完备且需要判断的局部使用 agent。自主性不是越高越好；权限、失败成本和可验证性共同决定允许的自主范围。

最小判定

- 能否根据观察改变下一步，而不是只按固定脚本执行？
- 是否能对外部环境产生受控行动？
- 是否存在预算、最大步数、成功条件和退出条件？
- 失败后能否暂停、恢复、求助或安全终止？

如果前两项为否，它更可能是生成式功能或 workflow；如果后两项为否，它还不是可运营的 Agent。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

核心术语表

Agent 工程中的中英文标准术语。

本页由 `catalog/glossary.yaml` 维护。核心区分包括：Agent 是自主闭环；Workflow 是预定义控制流；Trajectory 是完整运行轨迹；Guardrail 是自动边界控制；Human-in-the-loop 是人工审批或复核；MCP 面向工具与上下文连接；A2A 面向独立 Agent 协作；Durable execution 表示可持久化、等待和恢复的运行。

术语必须服从上下文。例如“memory”可能指当前对话历史、外部向量检索、结构化用户档案或任务状态，设计文档应指明具体语义、作用域和保留期限。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

成熟度模型

以证据而非演示效果划分 Agent 的五级成熟度。

| 等级 | 特征 | 进入下一阶段的证据 |

|---|---|---|

| L0 生成式功能 | 单轮输出，无环境行动 | 明确任务与可重复成功标准 |

| L1 受控助手 | 少量只读工具，人在每一步确认 | 离线用例集与工具正确率 |

| L2 有界 Agent | 多步闭环，有预算和沙箱 | 轨迹评测、恢复测试、风险门禁 |

| L3 生产 Agent | 持久状态、审批、可观测、SLO | 线上质量、成本、安全与事件数据 |

| L4 Agent 平台 | 多租户、策略中心、互操作、持续改进 | 跨团队治理和可移植评测 |

成熟度不能由模型名称决定。升级意味着新增可证明的控制和运营能力，而不仅是加入更多工具或更长上下文。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

阅读路径

为工程师、架构师、产品负责人和治理人员提供不同入口。

- 应用工程师：核心机制 -> 单 Agent 模式 -> 生命周期 -> 评测 -> 五个实验。
- 平台架构师：参考架构 -> 持久执行 -> MCP/A2A -> 可观测性 -> 安全治理。
- 产品负责人：定义与边界 -> 机会发现 -> 成熟度 -> 经济模型 -> Build or Buy。
- 安全与治理：威胁模型 -> 最小权限 -> 记忆安全 -> 事件响应 -> 生产就绪清单。
- 研究者：历史时间线 -> 推理/反思 -> Benchmark -> 开放研究问题 -> 未来信号。

不要从框架列表开始。先定义任务、环境、失败成本和验收证据，再选择实现。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

Agent 分类法

从自主性、时间跨度、环境、组织与风险五个轴描述 Agent。

单一“Agent/非 Agent”标签信息量过低。工程评审应至少记录五个轴：自主性从建议到代执行；时间跨度从单轮到跨天持久任务；环境从纯文本到浏览器、代码与物理世界；组织从单 Agent 到层级或网络式多 Agent；风险从只读到不可逆外部副作用。

分类的用途是决定控制。高自主、长时间、开放环境和高副作用任务，需要更强的身份、审批、沙箱、预算、恢复与审计。一个系统可以在能力上很先进，却因缺乏这些控制而处于低生产成熟度。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 02

第二章 Agent 从哪里来

现代 LLM Agent 汇合了经典理性 Agent、BDI、状态空间规划、强化学习、Foundation Models 与分布式 workflow。Transformer 和大规模预训练让自然语言成为通用决策接口，ReAct 等工作则把推理与环境行动交错起来。

历史提醒我们：今天的记忆、规划、目标冲突和信用分配问题并不新，只是接口和规模发生了变化。

stable

Foundation Models 转折

Transformer 与大规模预训练为何让通用 Agent 成为可能。

Transformer 提供了高效的序列建模基础，大规模预训练则让模型获得跨领域语言与知识能力。提示与上下文把一个固定参数模型临时配置为不同角色，结构化输出和函数调用又把自然语言决策接到软件系统。

关键变化是接口统一：过去每种任务都需要专用感知器、规划器和规则，现在模型可以在同一上下文中理解目标、读文档、生成代码并选择工具。但概率生成并未消除经典控制问题，反而要求外部运行时提供类型、权限、状态和验证。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

历史时间线

Agent 技术从经典模型到 2026 工程体系的关键转折。

- 1995 : BDI 将信念、愿望和意图带入可执行 Agent 讨论。
- 2017 : Transformer 奠定大规模通用序列模型基础。
- 2022 : Chain-of-Thought 与 ReAct 连接显式推理和环境行动。
- 2023 : 工具、反思、长期记忆、开放环境与 Agent Benchmark 集中发展。
- 2024 : 工程社区开始明确区分预定义 workflow 与动态 agent。
- 2025 : MCP、A2A 等协议推动工具与 Agent 的互操作。
- 2026 : 持久运行时、全轨迹评测、最小权限、人工控制和可运营性成为主战场。

结构化版本位于 `catalog/timeline.yaml`。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

evolving

LLM Agent 里程碑

从推理提示到工具、记忆、反思与环境交互。

2022 年的 Chain-of-Thought 与 ReAct 展示了分解问题及交错推理与行动；2023 年 Toolformer 探索工具使用学习，Generative Agents 展示记忆-反思-计划架构，Reflexion 使用语言反馈改进重试，Voyager 将技能库与开放环境探索结合。随后工程焦点从“能否行动”转为“能否可靠、经济、可治理地长期行动”。

这些工作不应被读成一条单向能力曲线。不同任务对规划深度、反思次数和记忆结构的收益不同；更多循环也可能带来成本、漂移和攻击面。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

stable

规划、搜索与控制

从状态空间搜索到模型驱动的动态规划。

规划是在动作模型和约束下寻找从当前状态到目标状态的路径。经典方法强调可枚举状态、动作前置条件和代价；LLM Agent 则常在不完整模型中生成候选步骤，再通过工具观察修正。

工程上应把“模型建议”与“执行控制”分开：模型负责提出下一步，运行时负责校验参数、权限、预算和状态转换。对路径稳定的任务，确定性 workflow 更可靠；对分支无法预先枚举的任务，才使用动态规划。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

强化学习遗产

奖励、探索、信用分配如何影响 Agent 设计。

强化学习把 Agent 描述为在环境中选择行动、获得回报并改进策略的主体。它贡献了状态、动作、奖励、策略、价值和探索等语言，也揭示了稀疏奖励、延迟信用分配和奖励黑客等难题。

多数生产 LLM Agent 不会在线更新模型权重，但会通过提示、记忆、策略规则或离线优化形成“外环学习”。Reflexion 一类方法用语言反馈影响后续尝试。安全边界是：生产反馈不能未经审查直接变成长期记忆或策略，否则错误与攻击会被放大。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

符号智能体与 BDI

经典 Agent 如何用信念、愿望和意图组织行动。

经典智能体把环境表示为状态，把行为表示为规则、计划或策略。BDI 模型进一步区分 Beliefs、Desires 与 Intentions：系统持有什么世界认知、希望达到什么状态、当前承诺执行什么计划。其价值不是拟人化，而是把目标冲突、承诺维持和重新规划变成可讨论的结构。

LLM Agent 通常没有严格的 BDI 解释器，却重新遇到相同工程问题：上下文是否等于信念、待办是否等于意图、何时放弃旧计划。显式任务状态和策略层能弥补纯文本上下文的不稳定。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 03

第三章 Agent 的解剖

模型是决策组件，Agent 系统还需要上下文、记忆、检索、工具、状态、终止控制和人机界面。上下文解决“当前该看什么”，记忆解决“哪些信息跨步骤或跨会话保留”，工具解决“如何作用于环境”。

这些能力必须由类型、来源、权限和生命周期连接，不能只堆叠在消息列表中。

stable

Agent Loop 与状态

解释计划、行动、观察、更新与终止的执行闭环。

典型循环是：读取目标和状态，选择模型与上下文，模型返回消息或工具调用，运行时验证并执行，观察结果写回轨迹，随后判断成功、继续、暂停或失败。真正的控制点位于循环外层，而不是提示词里。

状态至少分为会话消息、结构化任务状态、工具产生的环境状态和可恢复检查点。混在一个消息列表里会让重试、审计和恢复变得脆弱。运行时应设置最大步数、时间、Token、费用、工具次数和连续失败阈值，并让终止原因成为可观测字段。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

上下文工程

为每一步选择最小、相关、可信且有边界的上下文。

上下文工程不等于把更多文本塞进窗口。它负责选择指令、任务状态、用户偏好、检索证据、工具定义和历史摘要，并明确各自优先级与信任级别。

上下文越长，注意力竞争、过期信息和注入面越大。推荐把稳定策略保留在受控指令层，把业务事实放在带来源的数据层，把可变历史压缩成可追溯摘要。每次工具调用只暴露完成当前步骤所需的最小上下文。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

stable

目标、任务与环境

将模糊意图转成可执行、可验证、可终止的任务契约。

用户愿望通常不是可执行规格。任务契约需要目标、输入、允许行动、禁止行动、成功证据、预算、截止时间和升级路径。环境则定义可观察状态及行动如何改变它。

良好目标可被外部检查，而不是让模型自我宣告完成。比如“修复缺陷”应转换为复现测试失败、修改范围、测试通过和无未授权文件改动。开放环境必须标注不确定性；外部网页、邮件和文档既是数据源，也是潜在指令注入载体。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

人机协作

设计委托、可见进度、审批、纠错与控制权归还。

人在回路不应只在最终弹出“确认”。良好交互让用户在开始时理解能力和边界，运行中看到计划、证据与消耗，在关键副作用前获得具体审批，在失败时能够纠错、缩小范围或接管。

审批信息应说明对象、动作、影响、可逆性和替代方案。过多低价值确认会造成批准疲劳；过少则失去控制。系统应根据风险分层：安全只读自动执行，可逆写操作批量预览，不可逆或高影响动作逐项批准。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

experimental

学习与适应

区分会话内适应、记忆更新、提示优化与模型训练。

Agent 的“学习”可能只是当前会话调整，也可能是长期记忆、技能库更新、提示优化、检索索引变化或模型权重训练。设计必须明确哪一层发生变化、谁批准、如何回滚。

安全的改进闭环从生产轨迹采样，经人工或自动评分形成数据集，再离线比较候选变化，最后小流量发布。让生产 Agent 直接把成功经验写入全局策略，会引入数据投毒、局部过拟合和不可审计漂移。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

记忆

区分工作记忆、情节记忆、语义记忆和结构化用户状态。

工作记忆服务当前步骤；情节记忆记录过去事件；语义记忆沉淀稳定知识；程序性记忆保存技能或策略。

生产系统还需要结构化任务状态和用户档案，两者不应与自由文本混淆。

写入比读取更危险。每条长期记忆应有来源、租户、主体、有效期、敏感级别和删除能力。未经验证的外部内容不能自动变成高信任记忆。检索命中也不代表事实正确，模型仍需看到来源与时间。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

多模态与 Computer Use

Agent 如何通过视觉、语音和图形界面感知并行动。

Computer Use 把屏幕、DOM、键鼠和应用状态变成环境。它能覆盖缺乏 API 的长尾流程，却比结构化工具更脆弱：像素和布局会变化，界面文本可能注入指令，误点击可能产生真实副作用。

优先级应是 API/结构化工具、可访问性树或 DOM、最后才是纯视觉坐标。每次高风险操作前重新观察目标、校验窗口与账户、展示动作摘要并请求批准。截图与操作轨迹属于敏感审计数据，应限制留存。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

推理、规划与反思

何时分解、搜索、重试与反思，何时应保持简单。

规划可采用一次性计划、逐步计划、树搜索或由环境反馈驱动的滚动计划。反思则把失败原因转成下一次尝试的约束。它们的价值取决于任务是否可从中间反馈学习；对简单任务，额外循环往往只增加延迟和漂移。

工程上不应存储模型的私有推理，而应记录可审计的决策摘要、选择依据、工具参数和结果。重试必须改变策略或输入，否则只是重复消耗。达到失败阈值后应切换工具、缩小目标或请求人工介入。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

检索与 Grounding

让行动基于可追溯证据，而非模型记忆或搜索片段。

检索系统负责发现候选证据，Grounding 则要求输出或行动与证据建立可检查关系。典型流程包括查询改写、权限过滤、混合检索、重排、片段选择、引用和答案验证。

Agent 场景还需防止检索结果携带恶意指令。外部文本只能作为数据，不应覆盖系统策略。对于会触发副作用的结论，应从搜索摘要进入原始来源，核验时间、主体和适用范围。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

工具与 Function Calling

设计清晰、最小权限、可验证、幂等的 Agent 工具。

工具是 Agent 改变环境的能力边界。高质量工具有窄职责、清晰名称、严格参数 Schema、结构化结果、稳定错误语义和可区分的读写风险。不要用一个“万能 API”隐藏权限和副作用。

写操作应支持幂等键、预览、审批、超时和补偿；读操作应做作用域和租户过滤。工具结果必须区分业务失败、权限失败、暂时故障和参数错误，让模型能选择正确恢复策略。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 04

第四章 Agent 架构

可靠架构从最简单形态开始：增强调用、确定性 workflow、单 Agent 循环，再到确有多 Agent。控制面、执行面、数据面和保证面分离后，模型可以变化，而权限、状态和审计仍保持稳定。

多 Agent 的价值来自并行、专业隔离和上下文分区；没有这些需求时，它只会增加协调熵。

stable

反模式

识别无限循环、万能工具、无来源记忆和过度多 Agent 等常见失败。

- 无限自主循环：没有步数、预算或终止证据。
- 万能工具：一个高权限入口承担所有读写操作。
- 提示词安全：把权限控制写成“不要做坏事”。
- 无来源记忆：把模型总结直接当长期事实。
- 过度多 Agent：用角色数量掩盖职责和工具设计缺陷。
- 只看最终答案：忽略错误工具、越权尝试和偶然成功。
- Demo 驱动架构：只优化一条成功路径，没有恢复、审计和运营模型。

修复顺序通常是收紧任务、改进工具契约、补结构化状态与评测，最后才增加编排复杂度。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

图与事件驱动 Agent

用显式节点、边、事件和检查点组织复杂运行。

图结构把模型步骤、工具、审批和分支表示为可观察节点，使循环、并行和恢复比隐式 while loop 更清晰。事件驱动架构则让外部消息触发、暂停或继续任务，适合跨系统长流程。

图不应成为视觉化的意大利面。节点要对应稳定职责，边要表达真实状态转换，副作用要有幂等和重放策略。模型生成的临时计划可以作为数据，不应动态改写未经校验的生产执行图。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

多 Agent 模式

主管-专家、对等协作、辩论与市场式调度的适用边界。

多 Agent 的真实收益来自并行独立探索、专业权限隔离和上下文分区。常见模式是主管分派专家、对等交接、生成者-评审者，以及按能力发现远程 Agent。

代价包括消息膨胀、责任模糊、循环委派、共享状态冲突和更难的全局评测。每个 Agent 必须有明确所有权、输入输出契约、预算和停止条件；调度器应能检测循环并保留完整跨 Agent 轨迹。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

生产参考架构

一个可运营 Agent 平台的控制面、执行面、数据面和保证面。

参考架构分四面：控制面管理 Agent 定义、模型、策略、版本和发布；执行面运行循环、工具、沙箱和持久状态；数据面提供检索、记忆、密钥和业务系统；保证面覆盖身份、审批、评测、追踪、审计和事件响应。

入口先完成身份和风险分级，再创建有预算的运行。每个工具调用经过策略判定，写操作可暂停审批；轨迹与业务结果进入观测和评测，改进通过离线门禁重新发布。该分层避免把所有责任塞进提示词或单一框架。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

evolving

路由、Handoff 与编排

区分任务分类、工具式调用和控制权交接。

路由决定由谁处理请求；把专家当工具调用时，主 Agent 保留控制权；Handoff 则把后续对话和任务所有权交给另一个 Agent。三者必须在轨迹中可区分。

编排层维护能力目录、上下文裁剪、超时、重试、并发、预算和汇总。交接只传递必要信息，并带上来源与未解决问题；不要复制全部历史。跨组织交接还需要身份、授权和数据最小化。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

Sandbox 与运行时

将模型决策与受限文件、网络、代码和凭据环境隔离。

运行时负责工具注册、上下文构建、策略执行、状态持久化和轨迹记录；Sandbox 则限制不可信代码和工具的文件、网络、进程、资源与生命周期。两者共同构成模型之外的可信计算基。

默认拒绝网络和宿主文件，按任务挂载最小目录与短期凭据。保留资源上限、进程树清理、输出大小限制和超时。Sandbox 不能替代应用层授权：被允许运行的代码仍可能通过合法 API 产生越权副作用。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

单 Agent 模式

增强调用、路由、规划执行、反思与受控循环。

常见模式包括带检索和工具的增强调用、模型路由、规划-执行、评审-修订以及由环境反馈驱动的受控循环。优先使用一个清晰专长、有限工具集和明确输出契约的 Agent。

拆分前先问：失败是否来自上下文过载、工具语义差或评测缺失？若是，增加 Agent 数量只会隐藏问题。

单 Agent 更容易共享上下文、调试轨迹和控制成本，是默认起点。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

状态与持久执行

支持等待、恢复、重放和跨天任务的运行时基础。

长任务会遇到人工审批、外部回调、限流和机器故障，因此不能依赖进程内存。持久执行保存任务输入、当前状态、已完成副作用、检查点、等待原因和恢复令牌。

恢复语义应明确定义为至少一次或至多一次；关键写操作用幂等键和补偿事务。模型输出本身不是状态真相，结构化状态机才是。重放时必须避免重复发送邮件、扣款或发布内容。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

Workflow 与 Agent

根据路径可预测性选择确定性编排或动态决策。

Workflow 用代码定义路径，适合步骤已知、合规严格、需要稳定延迟的任务。Agent 让模型动态选择路径，适合信息不完备、分支难枚举、需要探索的任务。最可靠的系统往往把两者组合：确定性外壳控制权限和状态，Agent 只负责局部判断。

选择标准不是“智能程度”，而是错误代价与环境不确定性。若所有合法步骤能在设计时列出，就优先 workflow；若需要根据未知观察不断重规划，再引入 agent loop。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

Agent-to-Agent 协议

A2A 的能力发现、任务协作、消息与产物交换模型。

A2A 面向独立且可能不透明的 Agent 系统。参与者通过 Agent Card 描述能力与端点，围绕任务交换消息、状态和产物，而不要求共享内部记忆或工具。它适合跨框架、跨团队甚至跨组织协作。

工程重点是任务所有权、身份传播、数据边界、超时与取消。远程 Agent 的自然语言声明不能替代契约或授权；调用方应验证返回产物、限制委托链深度，并保留跨域关联 ID。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

experimental

Agent-UI 协议

将流式状态、计划、工具、审批和产物表达给用户界面。

Agent UI 不只是聊天流。界面需要接收运行开始、阶段进度、工具请求、审批中断、可恢复状态、部分产物、错误与完成事件。事件应有稳定 ID 和顺序语义，断线重连后能恢复。

协议层应区分模型文本与可信系统状态，避免模型伪造“已批准”或“已完成”。高风险动作使用结构化卡片显示对象、影响与权限。A2UI、AG-UI 等方向仍快速演进，核心知识库关注其共同语义而非绑定单一实现。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

发现、身份与授权

让 Agent 能找到能力，同时保持主体、权限和委托链可验证。

能力发现回答“谁能做什么”，身份回答“谁在请求”，授权回答“是否允许在此对象上执行此动作”。三者不能用模型描述互相替代。服务身份、最终用户、调用 Agent 和委托 Agent 应在审计中分开。

使用短期、受众限定、作用域最小的凭据；策略结合用户、Agent、工具、资源、动作和风险。多级委托必须限制深度并保留原始主体。机密只在执行边界注入，不能进入提示、长期记忆或普通轨迹。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

Model Context Protocol

MCP 如何标准化 AI 应用对工具、资源和上下文的访问。

MCP 定义客户端、服务器与宿主之间的能力协商，使工具和资源不必为每个模型平台重复适配。2026-07-28 规范强化了无状态核心、多轮请求、扩展、任务和授权能力，适合长期 Agent 工作流。

协议统一不等于安全自动成立。宿主仍需决定哪些服务器可信、向每个调用传递什么身份和上下文、哪些工具需要批准。对远程 MCP，应验证服务器身份、限制令牌受众、过滤工具元数据并记录完整调用轨迹。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

可移植性与兼容性

避免把 Agent 定义、状态和评测锁死在单一模型或框架。

可移植性包含工具协议、消息与状态 Schema、模型适配、追踪语义和评测数据。完全抽象往往压平厂商特性；完全绑定又提高迁移成本。合理边界是稳定业务契约加显式能力探测。

版本升级应运行同一离线评测集，比较质量、工具行为、延迟和成本。不要假设不同模型的系统指令、并行工具、结构化输出和上下文截断语义相同。协议版本与 Agent 定义版本都应进入轨迹。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 05

第五章 从原型到生产

完整生命周期从机会发现和威胁建模开始，经窄切片构建、轨迹评测、分阶段发布、生产观测、持续改进，最后退役。每一阶段都要有进入和退出证据。

生产化的标志不是 Demo 更流畅，而是任务可恢复、写操作幂等、版本可追踪、权限可撤销、事件可处置。

stable

01 发现与定界

判断问题是否值得使用 Agent，并形成任务与成功契约。

先定义用户、任务频率、当前流程、失败成本、数据与系统边界。Agent 适合步骤无法完全预定义、需要处理非结构信息、工具反馈能帮助下一步判断的任务。不适合没有客观成功标准或单次错误不可接受且无法审批的场景。

产物包括任务契约、基线流程、成功指标、风险分级、允许工具和不做清单。进入设计阶段前，应有至少 10 个真实代表案例，而不是只用理想化演示。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

02 设计与威胁建模

同时设计控制流、数据流、权限、失败路径与评测策略。

画出用户到模型、工具、数据和外部系统的信任边界。对每条边记录身份、数据类别、读写动作、可逆性和审计需求。明确哪些决策由代码、模型、策略引擎或人工承担。

威胁模型覆盖直接/间接注入、越权工具、数据外泄、记忆投毒、供应链和拒绝服务。设计评测集与架构同步进行，否则上线后才会发现无法判断系统是否改善。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

stable

03 构建与集成

从单 Agent 和窄工具集开始，建立类型化边界与确定性测试。

先实现一个端到端窄切片：清晰指令、少量高质量工具、结构化状态和显式终止。用 Fake Model 与工具替身测试控制流，再接真实模型。提示、工具 Schema、模型参数和策略均版本化。

所有副作用经统一执行网关，加入幂等、超时、审批和审计。不要在早期引入多 Agent，除非任务确实能独立并行或需要权限隔离。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

04 测试、评测与 Red Team

结合确定性测试、轨迹评分、人工复核和对抗场景。

测试分层：单元测试验证工具和策略；场景评测验证端到端任务；轨迹评分检查工具选择、参数、交接和违规；人工复核发现评分器遗漏；Red Team 验证注入、越权和外泄防线。

发布门禁使用版本固定的数据集、环境和评分标准。对非确定输出运行多次，报告分布而非单值。失败样本必须分类，不能只调提示追求总分。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

[stable](#)

05 部署与发布

用分阶段流量、版本绑定、回滚和权限门禁发布 Agent。

发布包应绑定 Agent 定义、模型、工具版本、策略、检索索引和评测结果。先影子运行，再内部用户、小流量和逐步扩展。每一阶段都有质量、成本、安全和人工升级阈值。

回滚不仅是代码回滚，还要处理已写入状态、已启动外部任务和新记忆。高风险工具默认关闭，通过配置和身份分组逐步开放。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

06 观测与运营

把轨迹、业务结果、成本和安全事件连接起来。

追踪记录运行、模型调用、工具、交接、护栏、审批和状态转换；指标覆盖成功率、人工接管、重试、延迟、Token、工具费用与风险事件。日志必须脱敏并支持按租户、版本和场景切片。

告警应面向可操作症状，如连续工具失败、费用异常、越权拒绝激增或任务卡死。运营手册定义暂停 Agent、撤销凭据、隔离记忆和恢复任务的步骤。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

07 改进与优化

用生产证据驱动质量、成本与延迟的可回归改进。

改进循环从失败聚类开始：任务定义、上下文、工具、模型、策略、检索或环境哪一层出了问题。先修确定性边界，再调整提示或模型。每次变化回放黄金集和历史失败集，并比较总体与关键分群。

成本优化包括缩短上下文、缓存稳定前缀、按步骤选择模型、减少无收益反思、批量或并行独立任务。任何成本下降都要与质量和风险指标一起评估。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

[stable](#)

08 退役与归档

安全停止 Agent、撤销权限并保留必要审计证据。

退役前盘点活跃任务、定时触发器、服务身份、API Key、MCP 连接、长期记忆和下游依赖。停止新任务后等待或迁移在途运行，再撤销凭据和网络权限。

根据法规与业务策略归档 Agent 定义、评测、发布记录、事件和必要轨迹，同时删除不再需要的用户数据。README 和能力目录应明确标记替代方案，避免“幽灵 Agent”继续被调用。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 06

第六章 评测与可观测

Agent 评测需要同时看结果和过程。一次幸运的正确答案，可能来自越权工具或错误数据；一次合理失败，也可能体现了良好校准和及时求助。轨迹、业务结果、成本和安全事件必须关联。

评测不是发布前的一次考试，而是从生产失败回流到离线数据集的持续改进系统。

evolving

Benchmark 全景

理解 AgentBench、GAIA、WebArena、SWE-bench 的测量对象与局限。

AgentBench 覆盖多环境交互，GAIA 面向通用助理现实问题，WebArena 评测网站操作，SWE-bench 用真实仓库 Issue 衡量软件工程能力。它们适合比较研究进展和发现失败类型，但不能替代业务评测。

基准分数受环境、工具、提示、采样和污染影响。报告时记录完整 harness、模型版本、运行次数和失败分类。生产决策应以内部代表任务与风险约束为主，公共 Benchmark 为辅。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

stable

能力与可靠性

区分偶尔能完成、稳定完成和在受扰条件下完成。

能力回答系统在理想条件下能否完成，可靠性回答它在输入变化、工具错误、长上下文和多次重复中是否仍能完成。报告 pass@1、重复成功分布、恢复率和最坏分群，而不是只展示最佳案例。

可靠性还包括校准：不确定时能否求助或停止。一个知道边界并升级的 Agent，通常比偶尔高分但自信越权的系统更适合生产。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

成本、延迟与质量

用 Pareto 前沿而非单指标比较 Agent 方案。

总成本包含模型 Token、工具、搜索、沙箱、存储、人工审批和失败返工。延迟拆为首响应、模型、工具等待、审批和尾延迟。质量要按任务价值和风险加权。

比较方案时绘制质量-成本-延迟 Pareto 前沿。便宜但需要大量人工返工并不经济；高分但 P99 超时也不可用。预算控制是运行时功能，超出时应降级、缩小任务或暂停。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

数据集与测试用例

建立可复现、分层、版本化且尊重隐私的 Agent 测试集。

每个用例包括初始状态、用户目标、可用工具、模拟环境、成功条件、禁止行为和评分说明。生产样本进入数据集前要脱敏、去重并获得适当使用权限。

版本记录新增、修正和删除原因。测试环境冻结外部依赖或使用可重放模拟器，避免网站变化污染结论。对非确定模型，多次运行并保存随机性与模型版本。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

评测策略

从业务目标反推任务集、评分器、阈值与发布决策。

先定义决策：是否发布、选择哪个版本、哪里需要人工。再为任务成功、过程合规、安全、延迟和成本设置指标与最低阈值。评分器可组合确定性断言、程序检查、模型评分和人工复核。

评测集应覆盖常规、边界、历史失败和对抗样本，并按真实流量加权。保留一部分盲测集，避免围绕公开样本过拟合。任何总分都必须能下钻到场景与失败类型。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

stable

人工评测

用专家判断补足自动评分器对价值、语境和风险的盲区。

人工评测适合主观质量、复杂事实核查、策略边界和新失败模式。评分指南应提供正反例、冲突处理和“不足以判断”选项；双人标注与一致性统计能暴露模糊标准。

人工不是无限容量的兜底。优先抽样高风险、低置信、评分器冲突和新版本变化样本，把结论沉淀为更好的确定性检查或用例。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

安全与对抗评测

验证注入、越权、外泄、记忆投毒与资源滥用防线。

对抗用例覆盖恶意用户输入、网页或文件中的间接注入、工具元数据欺骗、跨租户检索、记忆污染、凭据诱导和无限循环。目标不仅是模型拒绝，还要证明执行层策略无法被模型绕过。

记录攻击前置条件、预期控制、实际轨迹和残余风险。Red Team 结果进入回归集；修复后复测相邻攻击面，避免只阻断某个字符串。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

evolving

轨迹与工具使用评测

检查工具选择、参数、顺序、恢复与不必要动作。

最终结果可能偶然正确，但过程使用了错误数据、越权工具或不可接受成本。轨迹评测检查是否选择正确工具、参数是否满足 Schema、是否引用观察、是否重复调用、是否在失败后采取合理恢复。

不要求轨迹完全匹配一条黄金路径，因为复杂任务可能有多条合法方案。应评估关键不变量和禁止行为，并把可接受替代路径显式编码。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 07

第七章 安全、安全性与治理

工具、记忆和外部内容扩大了攻击面。真正的边界应存在于身份、授权、策略网关、沙箱和审批层，而不是模型承诺。Prompt Injection 不能被彻底“提示掉”，只能通过纵深防御限制影响。

治理最终要保留有意义的人类控制：知道系统在做什么，能暂停、拒绝、纠错并追责。

evolving

Agent 威胁模型

从主体、资产、信任边界和攻击路径系统分析风险。

盘点用户、Agent、模型提供商、工具、MCP 服务器、数据源和远程 Agent；标出凭据、个人数据、业务对象和外部副作用。威胁包括注入、越权、外泄、记忆投毒、身份混淆、供应链、资源耗尽与审计规避。每项风险记录前置条件、影响、预防、检测和恢复。模型拒绝只能是纵深防御的一层，真正权限必须由确定性策略与执行边界实施。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

evolving

Guardrails 与人类控制

在输入、输出、工具和高风险决策处组合自动控制与人工审批。

输入护栏阻断非法目标，工具护栏校验参数与权限，输出护栏处理泄露和政策问题。人工审批用于不可逆、高价值或语境复杂的动作。护栏结果要进入轨迹并支持恢复。

控制权应可见、及时且可操作：用户能暂停、取消、修改范围和拒绝单项动作。审批不能只显示模糊描述，而要展示对象、参数、影响与可逆性。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

身份、委托与供应链

管理非人身份、委托链和第三方工具依赖。

区分用户身份、Agent 工作负载身份、工具服务身份和远程 Agent 身份。委托令牌保留原始主体、限制能力和深度，避免“代表用户”退化为长期共享密钥。

第三方 MCP、插件、模型与依赖都是供应链。固定版本、验证来源、审查权限和更新差异；能力描述变化应触发重新批准。停用组件时撤销密钥并检查残留任务。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

Agent 事件响应

检测、限制、调查和恢复 Agent 引发的安全或业务事件。

预案包含一键暂停 Agent、禁用工具、撤销凭据、隔离记忆、终止在途任务和阻断远程连接。调查需保存版本、轨迹、审批、策略判定和外部系统日志。

恢复前确认副作用范围，必要时执行补偿。事后将事件转成评测用例、策略或工具修复，并审查相邻能力，避免只修复一个提示字符串。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

记忆与数据安全

防止跨租户泄露、长期投毒和超期保留。

记忆条目必须有主体、租户、来源、用途、敏感级别、有效期和删除路径。检索先做权限过滤，再做语义相似度；不要从共享向量库中取回无权查看的数据。

写入前验证事实与来源，低信任内容进入隔离区。用户应能查看、更正和删除长期记忆。训练、评测、日志与产品记忆的用途不同，不能默认为互相复用。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

权限与最小特权

按用户、Agent、工具、对象和动作缩小授权范围。

Agent 继承用户意图，不应自动继承用户所有权限。读与写、草稿与发送、查询与转账必须拆成不同能力。短期凭据限定受众、租户、资源和动作，并在运行结束后撤销。

授权在执行时重新判断，不能信任模型声称“用户已同意”。高风险工具支持预览和逐项批准，审批记录绑定具体参数，参数变化后重新审批。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

隐私、审计与合规

把数据最小化、可解释记录和法规义务纳入运行设计。

数据地图记录输入、派生数据、记忆、轨迹、模型提供商和保留期限。日志默认脱敏，敏感原文与普通指标分离访问。审计事件应能重建版本、身份、授权、工具和业务影响。

法规适用取决于地区、用途和风险分类。知识库提供工程原则，不替代法律意见。系统应支持数据访问、更正、删除、用途限制和自动决策的人类复核。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

Prompt Injection

防御用户、网页、文档和工具输出中的恶意指令。

直接注入来自用户，间接注入隐藏在 Agent 读取的网页、邮件、文档或工具结果中。仅靠提示告诉模型“忽略恶意内容”不足以形成边界。

防御组合包括数据与指令分层、来源标记、最小上下文、工具白名单、参数验证、敏感动作审批、输出过滤和持续对抗评测。被污染内容不能写入高信任记忆；跨边界复制数据前做策略检查。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

stable

沙箱与隔离

限制不可信代码、浏览器和工具的系统资源与网络范围。

沙箱默认无宿主凭据、无广泛网络、无永久文件；按任务挂载最小资源并设置 CPU、内存、磁盘、进程和时间限制。多租户运行必须隔离文件、缓存、日志和网络身份。

沙箱快照有助恢复和审计，但可能包含敏感数据，应加密并按策略删除。输出进入可信系统前仍需校验，不能因为代码在沙箱里运行就信任结果。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 08

第八章 生态与应用

模型、SDK、协议、运行时、记忆、观测和基础设施共同构成生态。最值得长期持有的资产是业务工具契约、结构化状态、评测数据和安全策略，而不是某个框架的表面 API。

应用成熟度与环境反馈密切相关：代码、数据和有界企业流程通常先于开放世界长期自治进入生产。

[stable](#)

部署基础设施

队列、容器、沙箱、状态存储与事件系统如何承载 Agent。

短请求可用无状态服务，长任务需要队列、工作器、检查点和回调。代码执行使用短生命周期沙箱，状态放在外部存储，凭据由密钥系统按运行注入。

基础设施要支持水平扩展、并发限额、取消、优雅停止和跨区恢复。模型 API 限流是容量规划的一部分，不能只看计算资源。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

框架与 SDK

比较代码优先、图编排、多 Agent 和托管式开发方式。

主流框架通常提供 Agent 定义、工具、状态、流式事件、交接、护栏和追踪。差异集中在控制权、持久执行、部署集成与可观测性。先做最小原型验证抽象是否匹配，而不是依据示例数量选择。

保留自有业务工具和评测数据，框架适配放在边缘。若简单 API 循环已满足需求，不必引入复杂图或多 Agent 框架。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

evolving

记忆与检索系统

比较向量、关键词、图、关系和事件存储在 Agent 中的角色。

向量检索适合语义相似，关键词适合精确实体，关系数据库保存结构化真相，知识图表达关系，事件存储保留时间序列。生产记忆通常组合多种存储。

选择标准包括权限过滤、更新一致性、来源、删除、时间衰减和可解释性。不要让向量库同时承担用户档案、任务状态和审计日志。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

模型与提供商

按任务能力、工具可靠性、成本、延迟和数据控制选择模型。

Agent 模型选择关注指令遵循、工具调用、长上下文、结构化输出、多模态、延迟、价格、地域和数据政策。一个运行可按步骤路由不同模型，但路由本身要评测。

不要把厂商排行榜直接映射到业务。使用同一任务集比较成功率、轨迹违规和总成本，并记录精确模型快照。模型升级视为代码变更，必须回归。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

可观测与评测工具

统一追踪、数据集、评分器、实验和线上监控。

工具应支持跨模型与框架的运行 ID、模型/工具 span、数据脱敏、反馈、数据集版本和实验比较。OpenTelemetry 等开放语义有助避免观测锁定。

采购时检查数据驻留、采样、成本、评分器复现和原始轨迹导出。漂亮的 trace UI 不能替代业务结果关联和发布门禁。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

运行时与 Agent 平台

承载会话、持久状态、沙箱、队列、策略与多租户运营。

运行时执行单个闭环，平台进一步管理定义、版本、部署、身份、配额、观测和治理。选择托管或自建取决于数据边界、工具网络、控制需求和运维能力。

关键评审项是恢复语义、状态导出、策略可插拔、追踪完整性、模型可替换和成本归因，而不是 Playground 体验。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

[stable](#)

技术选型指南

从任务、风险和可移植资产出发做 Build or Buy 决策。

先确定任务分解、工具网络、数据驻留、恢复、审批、SLO 与团队能力，再评估产品。用两周窄切片比较端到端成功、集成工作量、观测、成本和退出路径。

核心业务逻辑、工具契约和评测集保持自有；通用沙箱、队列和观测可优先复用。避免同时引入多个重叠框架。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

技术全景

用能力层和成熟度观察快速变化的 Agent 生态。

全景分为模型、SDK/框架、协议、工具与数据、记忆检索、运行时、观测评测和安全治理八层。结构化清单位于 `catalog/technologies.yaml`。

成熟度标记的是生产证据而非流行度。每次更新记录版本、来源和观察日期；从全景删除项目时保留变更记录，避免历史失真。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

工具、连接器与 MCP

从私有函数到开放工具协议的能力层。

连接器封装外部 SaaS 和数据库，MCP 提供统一发现与调用协议。无论接口形式，工具都要有清晰 Schema、身份传播、风险标签和错误分类。

工具目录不是越大越好。运行时按任务搜索并只暴露少量候选，减少误调用和上下文开销。第三方工具进入生产前经过权限、数据和供应链审查。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

浏览器与 Computer Use Agents

操作网站和桌面应用的能力、脆弱性与控制模式。

浏览器 Agent 适合跨站检索、表单和缺乏 API 的流程。优先使用结构化 DOM 和稳定选择器，视觉坐标作为补充。每次导航后重新确认页面和账户。

付款、发布、删除和发送等动作必须预览并批准。网页内容视为不可信数据，隔离其指令；下载文件先扫描并限制后续工具权限。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

Coding Agents

从代码问答走向仓库级理解、修改、测试与评审。

Coding Agent 通过文件、搜索、补丁、终端和测试形成强反馈闭环。可靠流程先复现问题，最小修改，运行相关测试，再审查 diff。工作树隔离和权限边界是基础。

风险包括命令注入、依赖供应链、密钥泄漏和破坏性操作。基准通过不代表可维护性，仍需代码审查、静态检查和仓库约定。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

客户运营 Agents

客服、销售与客户成功中的检索、建议和受控行动。

Agent 可整合账户、知识库和历史交互，生成回复、更新工单或执行有限补救。涉及承诺、退款、价格和隐私时使用规则与人工审批。

评测按问题解决率、事实正确、政策遵循、升级质量和客户结果分层，避免只优化回复流畅度。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

企业工作流 Agents

在 CRM、ERP、工单、邮件和文档之间执行受控业务流程。

企业场景的价值来自跨系统理解非结构输入并推进流程。稳定步骤保持 workflow，Agent 处理分类、信息补全、异常调查和建议。写回系统前应用身份、审批和幂等。

成功指标是周期、返工、人工负担和业务结果，不是对话数量。部署从只读建议和草稿模式开始。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

研究、数据与科学 Agents

多源检索、分析、代码执行与证据综合的闭环。

这类 Agent 分解问题、搜索一手资料、执行代码、检查数据质量并综合结论。产物必须保留查询、数据版本、代码、假设和引用，使结果可复现。

防止把搜索排名当证据、把相关性当因果或把模型生成数据混入真实数据。高影响科学结论需要领域专家复核。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

experimental

机器人与 Embodied Agents

把语言规划连接到真实或模拟物理环境。

具身 Agent 将感知、空间表示、规划、技能和低层控制结合。语言模型适合高层任务解释与技能选择，实时安全控制仍应由确定性控制器承担。

仿真到现实存在感知误差与动力学差异。物理急停、速度/区域限制、碰撞检测和人工监督不能依赖语言模型。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

语音与多模态 Agents

在实时音频、视觉和工具调用之间保持低延迟与可控交互。

实时 Agent 需要语音活动检测、打断、低延迟工具和多模态状态同步。语音确认应重复关键实体与数字，敏感动作转入结构化确认。

评测覆盖转写错误、噪声、口音、抢话、延迟和错误工具调用。录音与图像具有更高隐私成本，应显式告知并限制留存。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 09

第九章 2026：我们在哪里

截至 2026-09-30，结构化工具调用、有界 Coding Agent、检索与基础追踪已较成熟；持久运行、多 Agent 互操作、长期记忆治理和 Agent UI 正快速演进；开放世界长期自主仍属实验。

竞争重心正在从“模型能否行动”转向“平台能否可靠、经济、可治理地承载行动”。

evolving

2026-09-30 快照

对能力、工程、标准和治理成熟度的时间截面判断。

已较成熟：结构化工具调用、代码与检索 Agent、基础追踪、单 Agent 有界循环、人工审批。快速演进：持久沙箱、长期任务、MCP 任务扩展、多 Agent 互操作、Agent UI。仍实验性：开放世界长期自主、无监督自我改进、跨组织 Agent 经济和高风险物理自治。

行业正从框架竞争进入基础设施与保证竞争。MCP 2026-07-28 强化无状态核心、任务和授权；A2A 形成跨 Agent 协作语言；主流平台把 tracing、evals、guardrails、sandbox 和 durable state 放入一等能力。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

能力前沿

2026 年 Agent 在代码、搜索、计算机使用和长任务上的真实边界。

Agent 在有反馈和可验证产物的环境中进步最快：代码可测试、网页可观察、数据分析可执行。困难仍集中在模糊目标、长依赖链、隐藏状态、脆弱 UI 和跨域权限。

前沿演示不能直接外推生产可靠性。任务长度增加时，单步小误差会复合；环境变化和恢复能力往往比模型静态基准更决定结果。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

经济性与运营模型

从 Token 成本转向完整任务成本、人工负担与价值兑现。

完整成本包括模型、搜索、工具、沙箱、存储、观测、人工审批、失败返工和安全运营。Agent 的价值是提高完成率、缩短周期或扩大可处理范围，而不是单纯减少人力。

运营团队需要 Agent Owner、领域专家、平台、安全和评测协作。每个生产 Agent 都应有预算、SLO、值班路径、变更记录和退出机制。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

局限与瓶颈

可靠性、状态、环境、权限、评测和组织能力仍是主要约束。

主要瓶颈是错误复合、环境不稳定、隐式状态、长上下文污染、工具语义差、缺乏代表评测、注入与越权，以及成本尾部。多 Agent 会进一步放大协调和归因困难。

组织瓶颈同样重要：没有任务 Owner、领域标注、事件响应和权限治理时，技术能力无法转化为可信生产系统。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

evolving

生产成熟度

哪些 Agent 模式已能产生稳定价值，哪些仍需严格约束。

成熟应用多为有界任务：编码辅助、企业知识检索、客服草稿、数据分析和内部流程调查。它们有明确工具、可检查结果、人工升级和回滚路径。

涉及资金、医疗、法律、公共安全或大规模不可逆动作时，应将 Agent 限制为建议或分步审批。生产成熟度来自系统控制和运营证据，而非模型自主性。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

CHAPTER 10

第十章 Agent 将去向何处

近期方向是持久运行、策略化授权、评测驱动发布与开放互操作。中期可能出现组织级能力网络、专业 Agent 协作和更强长期个性化，但安全、责任和经济性可能限制开放程度。

预测必须附带可观察路标；产品发布数量不是采用和价值的替代指标。

experimental

自适应与自我改进 Agent

从轨迹中改进提示、技能、记忆和策略的受控闭环。

近期可行形态是外环改进：从轨迹发现失败，生成候选规则或技能，离线评测后由人批准发布。直接在线修改核心策略风险很高。

关键研究是防数据投毒、跨任务泛化、信用分配和安全回滚。若改进无法在保留集持续复现，就不应称为学习。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

speculative

自主性与人类控制

未来系统将动态调节自主级别，而非追求无条件自治。

自主性会按任务、用户、环境和实时风险动态变化：低风险自动执行，异常时缩小权限，高影响动作请求批准。控制界面将展示计划、证据、预算和可逆性。

衡量进步不应是“无人运行多久”，而是系统在更少干预下仍能保持目标一致、及时求助并限制最坏损失。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

speculative

未来 3-5 年

从单应用 Agent 走向组织级能力网络，但受治理和经济性约束。

可能出现组织级 Agent 身份、能力市场、跨企业任务协议、自动化评测生成和更强的个性化长期记忆。人类角色从逐步操作转向目标设定、例外处理、审计与制度设计。

这不是必然路径。责任归属、数据主权、安全事件和边际经济性可能推动生态回到更集中、受控的平台。判断应关注真实委托规模与事故数据。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

experimental

多 Agent 生态

能力发现、委托、协商与责任链可能形成的新系统层。

开放协议可能让专业 Agent 像服务一样被发现和组合。真正难点不是通信，而是身份、服务质量、价格、数据边界、委托深度和争议处理。

早期更可能在单组织或受信联盟中成熟，而非完全开放市场。互操作测试套件和责任可追踪性是关键领先指标。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

speculative

未来 12-24 个月

持久运行、策略执行、评测和开放互操作将成为基础能力。

高置信方向包括：长任务的暂停/恢复进入平台层；工具权限与审批统一策略化；trace 与 eval 驱动发布；MCP/A2A 形成更成熟目录与网关；模型按步骤动态路由；Agent UI 从聊天走向任务控制台。

如果主流企业仍停留在只读 Copilot、协议实现缺少互操作测试，或持久任务成本无法下降，上述速度判断需要下调。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

experimental

开放研究问题

长期可靠性、可校准规划、记忆安全和多 Agent 协调仍未解决。

核心问题包括：如何预测长轨迹成功率；如何在环境变化中校准不确定性；如何证明工具策略不被注入绕过；如何安全更新长期记忆；如何评测多 Agent 的涌现失败；如何把能力提升转化为可控自主性。

研究结果应在隐藏测试、真实环境和安全约束下复现，避免只优化静态 Benchmark。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

speculative

情景与路标

用可观察信号持续修正未来判断。

受控普及：企业以策略平台托管大量窄 Agent。路标是生产 SLO、事故数据和权限标准化。开放网络：跨组织 Agent 协作增长。路标是互操作认证、结算和责任机制。能力停滞：长任务可靠性与成本改善有限，产品回归 Copilot。

每季度更新路标，不用新产品发布数量代替真实任务完成率和运营证据。

[扩展阅读：仓库中的完整章节与持续更新（点击打开）](#)

speculative

标准与基础设施

协议、身份、策略、评测与追踪将组成 Agent Internet 的底座。

工具、Agent 和 UI 协议会继续分层，基础设施增加能力目录、策略网关、可信身份、任务队列和跨域追踪。标准成功取决于独立实现能否互通，而不是规范数量。

应关注认证、授权、任务恢复、错误语义和安全配置是否形成共享测试，而不仅是消息格式。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

CHAPTER 11

第十一章 工程行动手册

从一个窄任务开始：定义外部成功证据，限制三个以内工具，建立离线用例和 Fake Model，加入步数与费用预算，再接真实模型。先上线只读或草稿模式，再逐步开放写操作。

当失败出现时，先定位目标、上下文、工具、状态、环境或权限层，不要默认通过更大模型或更多 Agent 解决。

stable

Build or Buy 决策指南

根据差异化、控制、风险和运营成本决定自建边界。

自建适合核心业务逻辑、独特工具、严格数据边界和需要深度控制的场景；采购适合通用沙箱、观测、模型网关和低差异连接器。关键不是全买或全建，而是保持任务契约、工具 Schema 和评测数据可迁移。试点必须验证数据导出、版本控制、权限、SLO、价格上限和退出路径。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

evolving

案例分析模板

用统一结构分析 Agent 成功与失败案例。

每个案例记录用户与任务、原流程、环境与工具、架构、权限、评测、发布方式、线上结果、事件与教训。成功案例必须包含代价与失败边界；失败案例避免只归因于“模型幻觉”。

首版案例域包括 coding、浏览器操作、企业调查和客户运营，后续接受带可验证证据的社区贡献。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

evolving

失败图谱

按目标、上下文、模型、工具、状态、环境和治理定位问题。

失败分类：目标含糊；上下文缺失或污染；模型计划错误；工具选择或参数错误；状态陈旧；环境变化；权限或策略缺陷；恢复失败；评分器误判。先定位层次，再修复。

不要默认“换更大模型”。若错误是权限、Schema 或状态，模型升级不会形成确定性修复。

扩展阅读：[仓库中的完整章节与持续更新（点击打开）](#)

stable

学习路径

从闭环实验逐步学习工具、记忆、评测、安全与生产架构。

1. 运行 `examples/agent_loop`，理解循环和终止。
2. 运行 `tool_use`，观察 Schema 与工具错误。
3. 运行 `memory`，区分短期状态和长期事实。
4. 运行 `eval_harness`，把成功与轨迹变成分数。
5. 运行 `secure_tool_gateway`，体验策略与审批。
6. 用自己的窄任务替换 Fake Model 输入，先保留离线回归。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

stable

生产就绪检查表

上线前检查任务、工具、状态、评测、安全、观测和运营。

- 目标、成功和终止条件可外部验证。
- 工具最小权限，写操作幂等且可审批。
- 状态可恢复，重试不会重复副作用。
- 代表、边界、历史失败和对抗用例通过门禁。
- 轨迹脱敏并关联业务结果、成本和版本。
- 有暂停、撤权、隔离记忆和事件响应手册。
- 有 Owner、SLO、预算、升级与退役路径。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

stable

参考蓝图

从入口到策略、运行时、工具、状态、评测和运营的生产路径。

请求经身份与风险分类进入编排器；编排器加载版本化 Agent、最小上下文和工具；策略网关验证每次行动；持久运行时保存状态；沙箱执行不可信代码；追踪连接业务结果；评测服务决定发布。

从单 Agent、三个以内工具和一个端到端指标开始。所有写操作经过统一网关，高风险动作可暂停审批。

扩展阅读：仓库中的完整章节与持续更新（[点击打开](#)）

参考资料 1/2

[russell-norvig-aima](#)

Artificial Intelligence: A Modern Approach

<https://aima.cs.berkeley.edu/>

[rao-georgeff-bdi](#)

BDI Agents: From Theory to Practice

<https://cdn.aaai.org/ICMAS/1995/ICMAS95-042.pdf>

[sutton-barto-rl](#)

Reinforcement Learning: An Introduction

<http://incompleteideas.net/book/the-book-2nd.html>

[transformer](#)

Attention Is All You Need

<https://arxiv.org/abs/1706.03762>

[cot](#)

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

<https://arxiv.org/abs/2201.11903>

[react](#)

ReAct: Synergizing Reasoning and Acting in Language Models

<https://arxiv.org/abs/2210.03629>

[toolformer](#)

Toolformer: Language Models Can Teach Themselves to Use Tools

<https://arxiv.org/abs/2302.04761>

[generative-agents](#)

Generative Agents: Interactive Simulacra of Human Behavior

<https://arxiv.org/abs/2304.03442>

[reflexion](#)

Reflexion: Language Agents with Verbal Reinforcement Learning

<https://arxiv.org/abs/2303.11366>

[voyager](#)

Voyager: An Open-Ended Embodied Agent with Large Language Models

<https://arxiv.org/abs/2305.16291>

[agentbench](#)

AgentBench: Evaluating LLMs as Agents

<https://arxiv.org/abs/2308.03688>

[gaia](#)

GAIA: a benchmark for General AI Assistants

<https://arxiv.org/abs/2311.12983>

[webarena](#)

WebArena: A Realistic Web Environment for Building Autonomous Agents

<https://arxiv.org/abs/2307.13854>

参考资料 2/2

[swebench](#)

SWE-bench: Can Language Models Resolve Real-World GitHub Issues?

<https://arxiv.org/abs/2310.06770>

[anthropic-effective-agents](#)

Building Effective Agents

<https://www.anthropic.com/engineering/building-effective-agents>

[anthropic-evals](#)

Demystifying Evals for AI Agents

<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

[anthropic-trustworthy](#)

Trustworthy agents in practice

<https://www.anthropic.com/research/trustworthy-agents>

[openai-agents-sdk](#)

Agents SDK

<https://developers.openai.com/api/docs/guides/agents/sdk>

[openai-agent-evals](#)

Evaluate agent workflows

<https://developers.openai.com/api/docs/guides/agent-evals>

[mcp-spec](#)

Model Context Protocol Specification 2026-07-28

<https://modelcontextprotocol.io/specification/2026-07-28>

[a2a-spec](#)

Agent2Agent Protocol Specification 0.3.0

<https://a2a-protocol.org/v0.3.0/specification/>

[google-adk-lifecycle](#)

Agents CLI Development Guide

<https://google.github.io/agents-cli/guide/development/>

[owasp-agent-security](#)

AI Agent Security Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html

[nist-ai-rmf](#)

Artificial Intelligence Risk Management Framework

<https://www.nist.gov/itl/ai-risk-management-framework>

[eu-ai-act](#)

Regulation (EU) 2024/1689

<https://eur-lex.europa.eu/eli/reg/2024/1689/oj>